INFORME DE LABORATORIO #3

Gramáticas Libres de Contexto

Antecedentes

Antecedente	Información
Asignatura	Teoría de la Computación - INFO1148
Carrera	Ingeniería Civil en Informática
Docente	Marcos Lévano
Integrantes	Jose Jimenez Saavedra (jefe de grupo); Demian Quezada Pasten (integrante).
Fecha	24 de octubre de 2026

1. Introducción

El objetivo de este laboratorio es diseñar y validar una Gramática Libre de Contexto (GLC) para un lenguaje de programación simplificado. Se identifican sus terminales, no terminales y símbolo inicial, se definen producciones para programas con asignaciones y salidas, y se comprueba su coherencia mediante derivaciones por la izquierda, un árbol de derivación y el análisis de cadenas válidas e inválidas. Finalmente, se incorpora una declaración de variables y se verifica formalmente su integración. El desarrollo sigue el enunciado, la plantilla y la rúbrica proporcionados, en ese orden de prioridad.

2. Desarrollo

2.1 Pregunta 1 – Componentes de la gramática

a) Símbolos terminales

Se distingue el análisis léxico, que reconoce lexemas, del análisis sintáctico, que organiza tokens. La GLC principal G opera sobre un alfabeto finito de clases de tokens. Un lexema es la escritura concreta de un token: por ejemplo, `temporal` pertenece a la clase ID y `1.345` a NUM.

Categoría	Elementos identificados
Palabras reservadas	`Begin`, `End`, `Programa`, `salida`; terminales literales.
Operadores	Asignación `=`; aritméticos `+`, `-`, `*`, `/`, `%`; terminales literales.
Símbolos especiales	`#` termina una instrucción. `_` forma parte de identificadores y `.` de reales: son caracteres léxicos, no tokens independientes de G.
Identificadores	Clase terminal ID, reconocida por la estructura `letra (letra | dígito | _)*`. Debe comenzar con letra y no ser una palabra reservada.

Categoría	Elementos identificados
Números	Clase terminal NUM: uno o más dígitos, seguidos opcionalmente de punto y uno o más dígitos. Incluye `0`, enteros positivos y reales sin signo como `1.345`.

`ID(z)` significa un token terminal ID con lexema `z`; no es un terminal adicional ni una producción distinta. Esta anotación se usa en las derivaciones y en el árbol. Lo mismo se aplica a `NUM(20)`. La lectura de los lexemas recupera el texto del programa.

b) Símbolos no terminales

```
V = {<programa>, <lista_instrucciones>, <instruccion>,
     <asignacion>, <asignacion_simple>, <asignacion_operacion>,
     <expresion>, <operando>, <operador>, <salida>, <id>, <numero>}
```

Todos aparecen como lado izquierdo de una producción y son alcanzables desde `<programa>`. Sus funciones se explican en 2.2 b).

c) Símbolo inicial

```
S = <programa>
```

Representa un programa completo: apertura, nombre, al menos una instrucción y cierre. No cambia al derivar fragmentos: las derivaciones desde `<instruccion>` muestran subárboles del programa, no un segundo símbolo inicial global.

d) Definición formal de la gramática

```
G = (V, Σ, P, S)
Σ = {Begin, Programa, End, salida, =, +, -, *, /, %, #, ID, NUM}
S = <programa>
P = producciones R1 a R12 de la sección 2.2 a).
```

V es exactamente el conjunto de 2.1 b); Σ contiene las clases terminales y los tokens literales; P establece las sustituciones permitidas; S determina qué es un programa completo. V y Σ son disjuntos. Cada lado izquierdo de P contiene un único no terminal, por lo que G es libre de contexto.

Formalmente, `L(G) = {w ∈ Σ* : <programa> ⇒* w}`. El lenguaje fuente aceptado es `{texto : lex(texto) está definido y sus clases de tokens pertenecen a L(G)}`. Esta distinción evita confundir infinitos lexemas con un alfabeto sintáctico infinito. Para una instrucción aislada se verifica `<instruccion> ⇒* w`; por sí sola no pertenece a L(G), pues le falta el envoltorio del programa.

e) Análisis

No corresponde enumerar identificadores ni números: se pueden construir cadenas de longitud arbitraria y, por tanto, sus conjuntos de lexemas no están acotados. Una regla estructural finita describe todos los casos admitidos. ID y NUM son dos categorías terminales de G; sus valores se reconocen mediante las reglas generales de la siguiente sección. Los identificadores y los valores numéricos no se confunden con no terminales: `<id>` y `<numero>` son los no terminales que insertan esas categorías en la sintaxis.

2.2 Pregunta 2 – Construcción de la Gramática Libre de Contexto

a) Producciones propuestas

En la tabla, `|` separa alternativas y no representa un terminal del lenguaje.

Regla	No terminal	Producción
R1	`<programa>`	`Begin Programa <id> <lista_instrucciones> End`
R2	`<lista_instrucciones>`	`<instruccion> <lista_instrucciones> | <instruccion>`
R3	`<instruccion>`	`<asignacion> | <salida>`
R4	`<asignacion>`	`<asignacion_simple> | <asignacion_operacion>`
R5	`<asignacion_simple>`	`<id> = <operando> #`
R6	`<asignacion_operacion>`	`<id> = <expresion> #`
R7	`<expresion>`	`<operando> <operador> <operando>`
R8	`<operando>`	`<id> | <numero>`
R9	`<operador>`	`+ | - | * | / | %`
R10	`<salida>`	`salida <id> #`
R11	`<id>`	`ID`
R12	`<numero>`	`NUM`

R2 genera listas no vacías: una instrucción mediante el caso base o más mediante la alternativa recursiva. R4 separa explícitamente la asignación de un valor y la asignación de una operación; R3 y R10 distinguen ambas de la salida. R7 exige la forma infija operando-operador-operando. Cada asignación admite una operación binaria como máximo; no se requieren precedencia, paréntesis ni operadores unarios en el lenguaje base.

La especificación léxica complementaria se puede expresar mediante estas reglas estructurales; sus auxiliares pertenecen al nivel léxico, no a V ni a P de G:

```
<lex_id> → <letra> <cola_id>
<cola_id> → <letra> <cola_id> | <digito> <cola_id>
          | _ <cola_id> | ε
<lex_numero> → <digitos> | <digitos> . <digitos>
<digitos> → <digito> <digitos> | <digito>
<letra> → c, para cada carácter alfabético Unicode c
<digito> → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

`<lex_id>` construye la palabra completa; `<cola_id>` permite su continuación, incluida la vacía; `<letra>` selecciona una letra; `<digito>` selecciona un dígito; `<digitos>` exige una secuencia no vacía; `<lex_numero>` permite entero o real con dígitos a ambos lados del punto. El enunciado no fija un alfabeto de letras: se adopta explícitamente el conjunto finito de caracteres alfabéticos Unicode. Los ejemplos exigidos usan letras latinas básicas.

El analizador reconoce la palabra completa más larga y da prioridad a las reservadas exactas, con distinción de mayúsculas y minúsculas. Así, `salida` no es ID, pero `salida2` sí. No hay signo, notación exponencial ni coma decimal en NUM; `-5` no es un literal admitido. El cero se acepta porque aparece expresamente en el enunciado. No se impone una restricción adicional a los ceros iniciales.

Se ignoran espacios, tabulaciones y saltos de línea entre tokens: `z=x+y#` y `z = x + y #` generan los mismos tokens. Esto no une palabras separadas: `x y` sigue siendo dos ID. El PDF incluye el comentario

`/* operaciones y salidas */`; se trata como comentario léxico de bloque no anidado y se ignora antes de aplicar G. Un comentario sin cierre es un error léxico. Esta convención permite representar el ejemplo completo sin introducir comentarios como instrucciones.

b) Explicación de los no terminales

No terminal	Función dentro de la gramática
`<programa>`	Genera el encabezado, el nombre, el cuerpo no vacío y el cierre del programa.
`<lista_instrucciones>`	Genera una o más instrucciones mediante recursión y un caso base.
`<instruccion>`	Selecciona una asignación o una salida; la extensión añade declaración.
`<asignacion>`	Distingue asignación simple y asignación con operación binaria.
`<asignacion_simple>`	Asigna un único operando a un identificador y exige el terminador #.
`<asignacion_operacion>`	Asigna el resultado de una expresión binaria y exige #.
`<expresion>`	Sitúa un operador entre exactamente dos operandos.
`<operando>`	Permite un identificador o un número en cada posición de valor.
`<operador>`	Selecciona uno de los cinco operadores aritméticos admitidos.
`<salida>`	Genera salida seguida de un identificador y del terminador #.
`<id>`	Inserta un token ID cuyo lexema cumple la regla general de identificadores.
`<numero>`	Inserta un token NUM, entero o decimal sin signo, reconocido por el léxico.

c) Verificación de la gramática

Estructura del programa de referencia	Producciones y comprobación
`Begin Programa operativo_01_02 ... End`	R1 exige las tres palabras del envoltorio; R11 inserta el ID cuyo lexema empieza con `o` y continúa con letras, guiones bajos y dígitos.
`x = 1.345 #`	R3, R4 y R5 eligen asignación simple; R8 y R12 insertan NUM. La regla léxica reconoce el punto y la parte fraccionaria.
`y = 20 #`, `temporal = 0 #`	La misma ruta admite un entero positivo y cero.
`z = x + y #`, `z2 = x + y #`	R6 y R7 generan dos operandos ID; R9 selecciona `+`. El dígito final de `z2` es válido.
`h = z - x #`, `h2 = z - x #`	R6, R7, R8, R9 y R11 admiten resta y los identificadores indicados.
`k = h * y #`, `t = k / z #`	R9 permite `*` y `/`, cada uno entre dos operandos ID.
`m = k % 10 #`	R9 selecciona `%`; R8 admite ID a la izquierda y NUM a la derecha.
`temporal = x #`, `x = y #`, `y = temporal #`	R5 permite copiar un ID mediante R8 y R11.
Las siete instrucciones `salida ... #`	R10 exige la reservada, un ID y el terminador.
Las veinte instrucciones del cuerpo	R2 aplica 19 veces la alternativa recursiva y una vez el caso base.

El programa completo se encuentra en `evidencias/programa_referencia.txt`, incluido el comentario del PDF, y es aceptado por el verificador. Cada una de sus veinte instrucciones se comprueba además por separado. La equivalencia de las dos escrituras con y sin espacios también se verifica. Estas comprobaciones son sintácticas: no evalúan resultados, declaraciones previas ni posibles divisiones por cero.

2.3 Pregunta 3 – Derivaciones y árbol sintáctico

a) Derivación por la izquierda para z = x + y

La raíz local es `<instruccion>`. En cada flecha se sustituye exclusivamente el no terminal situado más a la izquierda. Se conservan las anotaciones de lexema en los terminales ID para mostrar el texto concreto.

```
   <instruccion>
⇒ <asignacion>
⇒ <asignacion_operacion>
⇒ <id> = <expresion> #
⇒ ID(z) = <expresion> #
⇒ ID(z) = <operando> <operador> <operando> #
⇒ ID(z) = <id> <operador> <operando> #
⇒ ID(z) = ID(x) <operador> <operando> #
⇒ ID(z) = ID(x) + <operando> #
⇒ ID(z) = ID(x) + <id> #
⇒ ID(z) = ID(x) + ID(y) #
```

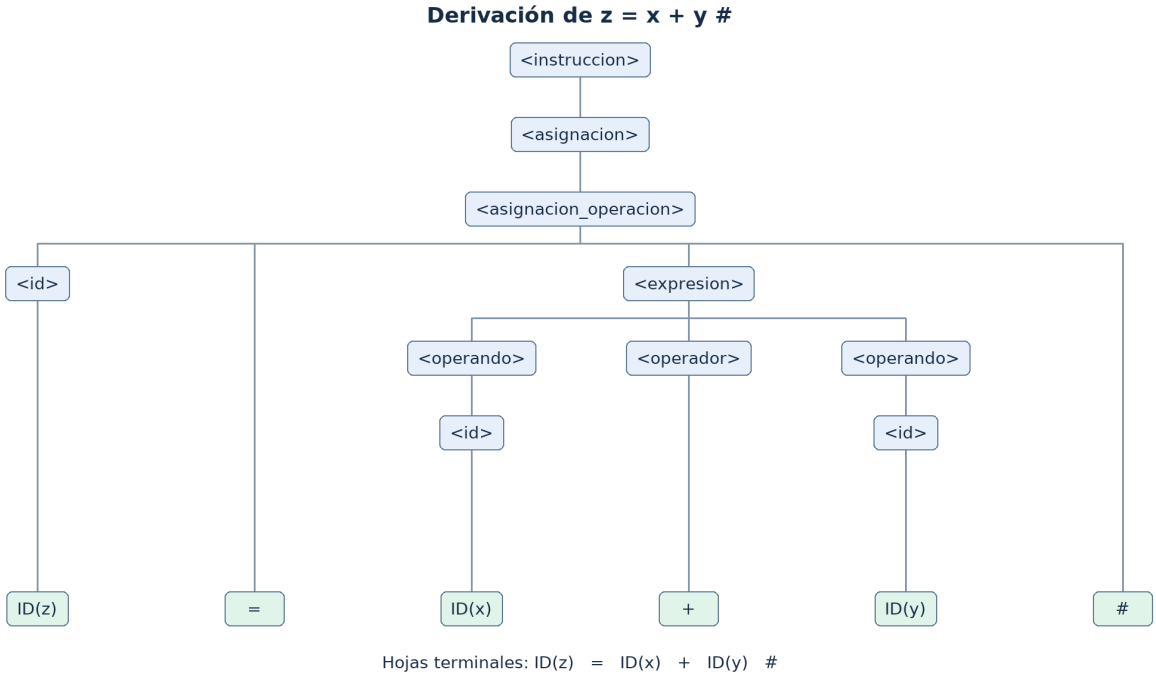
Las reglas aplicadas, en orden, son R3, R4, R6, R11, R7, R8, R11, R9, R8 y R11. El último renglón contiene únicamente terminales. Al leer sus lexemas se obtiene `z = x + y #`; esa lectura no es una producción adicional.

b) Derivación para salida z

```
   <instruccion>
⇒ <salida>
⇒ salida <id> #
⇒ salida ID(z) #
```

Se aplican R3, R10 y R11 al no terminal más a la izquierda. La lectura de las hojas terminales es `salida z #`. La regla R10 introduce la estructura de salida completa.

c) Árbol de derivación



La fuente editable está en `arbol_derivacion.mmd`; `evidencias/arbol_derivacion.svg` contiene una exportación vectorial del mismo árbol. La imagen incluye todos los nodos, sin abreviar las expansiones de ID.

d) Explicación del árbol

La raíz `<instruccion>` representa el fragmento que se deriva. Los nodos internos son no terminales y cada grupo ordenado de hijos reproduce exactamente el lado derecho de una producción de P. La raíz selecciona `<asignacion>`, que selecciona `<asignacion_operacion>`; esta genera `<id>`, `=`, `<expresion>` y `#`. La expresión se descompone en dos operandos y un operador, y los operandos se convierten en identificadores.

Las hojas son los terminales `ID(z)`, `=`, `ID(x)`, `+`, `ID(y)` y `#`. Las anotaciones indican lexemas de una misma clase ID. Su lectura de izquierda a derecha produce `z = x + y #`. El árbol muestra la jerarquía de la derivación; la derivación por la izquierda fija el orden en que se expanden sus nodos internos. El inicio global continúa siendo `<programa>`: este árbol puede insertarse como una instrucción del cuerpo mediante R1 y R2.

e) Validación de cadenas

Se evalúan como instrucciones, es decir, desde `<instruccion>`.

Cadena	¿Puede generarse?	Justificación
`x = 20 #`	Sí	R3 → R4 → R5 produce ID = operando #; R8 → R12 permite NUM(20), y R11 permite ID(x).
`z = x * y #`	Sí	R3 → R4 → R6 selecciona asignación con expresión; R7 exige dos operandos, R9 selecciona * y R8 → R11 genera ambos ID.
`salida temporal #`	Sí	R3 → R10 produce `salida <id> #`; R11 reconoce ID(temporal), cuyo lexema cumple la regla de identificadores.

2.4 Pregunta 4 – Validación y detección de errores

Los casos 1 a 7 se evalúan como fragmentos de instrucción y el caso 8 como programa completo. Por ello, «Sí» en los tres primeros significa que la cadena es una instrucción admisible dentro de un programa; no se afirma que derive desde S sin encabezado ni cierre. Las barras `/` del caso 8 del PDF separan líneas visualmente y no son tokens del programa. En el caso 5, `z=+xy#` del archivo de instrucciones y `z = + x y #` del PDF son inválidos; se informa la diferencia léxica.

Caso	Cadena	¿Pertenece al lenguaje?	Justificación / error detectado
1	`x = 10 #`	Sí, como instrucción	R3, R4 y R5 admiten asignación simple; R11 reconoce x y R8 → R12 reconoce 10. Incluye = y #.
2	`z=x+y#`	Sí, como instrucción	R6 y R7 generan dos operandos ID separados por +, admitido por R9. Los espacios son irrelevantes entre estos tokens.
3	`salida z #`	Sí, como instrucción	R3 y R10 generan salida con ID(z) por R11 y terminador #.
4	`x 10 #`	No	Falta = después de ID(x), exigido tanto por R5 como por R6. Corrección posible: `x = 10 #`.
5	`z=+xy#`	No	+ aparece donde R5/R6 exigen el primer operando. Además, `xy` se reconoce como un solo ID, no como x e y. En `z = + x y #` también falta un primer operando antes del operador; corrección posible: `z = x + y #`.
6	`salida #`	No	R10 exige `<id>` entre `salida` y #; falta ese identificador. Corrección posible: `salida z #`.
7	`x = 20`	No	Falta el terminador #, obligatorio en R5. Corrección: `x = 20 #`.
8	`Begin Programa prueba x = 5 # salida x # End`	Sí, como programa completo	R1 genera envoltorio y nombre ID(prueba); R2 genera dos instrucciones: R5 para la asignación y R10 para la salida. Ambos terminadores y End están presentes.

Análisis general

Los errores se explican por posiciones obligatorias en las producciones. Las asignaciones requieren = entre el destino y el valor; una expresión binaria requiere un operando antes y después del operador; una salida necesita un ID; toda instrucción termina en #. No basta con que una cadena contenga símbolos permitidos: debe respetar su orden y completar la estructura elegida. El reconocimiento léxico tampoco divide arbitrariamente `xy` en dos variables. Estas observaciones permiten detectar el punto concreto del fallo sin atribuir al análisis sintáctico tareas de evaluación semántica.

3. Extensión de la gramática

Construcción seleccionada: C. Declaración de variables.

Nueva sintaxis propuesta: una palabra de tipo, un identificador y `#`, por ejemplo `entero x #` o `real temperatura #`. Se incorpora una sola construcción con dos tipos disponibles. Facilita distinguir la declaración de una asignación sin modificar las expresiones originales.

Elemento	Modificación
Nuevos terminales	`entero`, `real`; son reservadas en la gramática extendida.
Nuevos no terminales	`<declaracion>` representa la instrucción completa; `<tipo>` selecciona el tipo declarado.
Nuevas producciones	R3 añade la alternativa `<declaracion>`; R13: `<declaracion> → <tipo> <id> #`; R14: `<tipo> → entero | real`.

Formalmente:

```
G' = (V', Σ', P', S')
V' = V ∪ {<declaracion>, <tipo>}
Σ' = Σ ∪ {entero, real}
P' = P ∪ {<instruccion> → <declaracion>,
          <declaracion> → <tipo> <id> #,
          <tipo> → entero, <tipo> → real}
S' = <programa>
```

El léxico de G' reserva `entero` y `real`, evitando que se utilicen como identificadores. Es un cambio explícito: textos que antes usaban esas palabras como nombres dejan de admitirse bajo G'. El programa de referencia no las usa como nombres y sigue siendo válido. No se agrega una exigencia de declaración previa, unicidad ni compatibilidad de tipos: esas verificaciones serían semánticas.

Cadena válida: `entero x #`, porque R13 exige tipo, ID y terminador. También se comprueba `real temperatura #`.

Cadena inválida: `entero 2x #`, porque 2x no es un identificador: comienza con un dígito. La secuencia de tokens NUM(2), ID(x) no puede ocupar el único `<id>` de R13. Corrección posible: `entero x2 #`.

Derivación de ejemplo por la izquierda:

```
   <instruccion>
⇒ <declaracion>
⇒ <tipo> <id> #
⇒ entero <id> #
⇒ entero ID(x) #
```

Se aplican la nueva alternativa de R3, R13, R14 y R11. La lectura terminal produce `entero x #`. Integración como programa completo válido:

```
Begin Programa prueba
entero x #
real temperatura #
x = 5 #
salida x #
End
```

R1 conserva el envoltorio y R2 permite intercalar declaraciones, asignaciones y salidas. Las comprobaciones automáticas aceptan este programa con G' y lo rechazan con G.

4. Conclusiones

La construcción de la gramática permitió distinguir los terminales que aparecen en una secuencia de tokens de los no terminales que describen su organización. Las reglas léxicas caracterizan identificadores y números sin enumerar sus posibles valores, mientras las producciones sintácticas establecen la estructura de programas e instrucciones. El símbolo inicial fija el alcance de un programa completo, y una derivación desde un no terminal interno explica un fragmento sin cambiar esa definición.

Las derivaciones por la izquierda y el árbol expresan las mismas producciones desde perspectivas complementarias: secuencia de sustituciones y jerarquía de componentes. La validación identificó ausencias y posiciones incorrectas de símbolos obligatorios. La declaración de variables mostró que una extensión debe actualizar formalmente terminales, no terminales y producciones, además de probarse dentro de programas completos. Mantener una única gramática base y una extensión explícita permitió conservar la coherencia entre definición, ejemplos, árbol y pruebas.

5. Consideraciones de entrega

Se adjuntan el informe en Markdown y PDF, la gramática, el árbol editable en Mermaid, exportaciones PNG y SVG, los tres PDF fuente y un verificador reproducible en Python sin dependencias externas. No se utilizó JFLAP, por lo que no corresponde adjuntar un archivo de esa herramienta. La estructura y numeración conservan la sección 5 presente en la plantilla original, que exige entregar PDF.

Repositorio: https://github.com/Pamtom21/LAB-3-TC

Fuentes principales: `documentos/Laboratorio_3.pdf` (objetivo, lenguaje y preguntas); `documentos/Formato_Informe_LAB_3.pdf` (estructura, antecedentes y formato de entrega); `documentos/Rubrica_Laboratorio_3.pdf` (corrección formal y criterios de calidad). No se utilizaron fuentes teóricas externas para reemplazarlas. La guía de ejecución se conserva en `documentos/LAB-3-TC_Codex_Instrucciones.md`.

Anexo de evidencias

Evidencia	Ubicación
Programa original completo	`evidencias/programa_referencia.txt`
Árbol y fuente editable	`arbol_derivacion.mmd`, `evidencias/arbol_derivacion.png` y `.svg`
Resultados automáticos y revisión de la rúbrica	`evidencias/validacion.txt`, `evidencias/revision_rubrica.md`
Gramática y derivaciones verificables	`validacion/gramatica.json`, `validacion/derivaciones.json`, `validacion/verificar.py`

La revisión de la rúbrica documenta cobertura y evidencia; la calificación corresponde al docente.